Graph-Structured Models, Their Algorithms, and the Properties That Referee Them

M. J. Wilson

January 1, 2025

Abstract

A self-contained account of the three problem classes this project supports, the algorithms that solve them, and—the part a methods section usually omits—the mathematical properties each algorithm is validated against, and why those properties are the right referees. Written for a reader with a scientific and performance-computing background but no background in phylogenetics or statistical physics. It describes *algorithms*, not an implementation: no result here depends on how any of it is coded, and the companion paper carries the results themselves.

Contents

1 Notation

One symbol, one meaning, across this document and the companion paper.

Symbol	Meaning
$\mathcal{T}$	a tree topology
$\mathcal{G}$	a general undirected graph
$\mathcal{A}$	the state alphabet, $\lvert\mathcal{A}\rvert = k$
ρ	the root of a rooted tree
$\mathcal{N}(\cdot)$	the neighbourhood a discrete move set reaches in one step
$\mathbf{Q}$	a rate matrix; $\mathbf{P}(t) = \exp(\mathbf{Q}t)$ its transition matrix
$\mathbf{S}$	a symmetric exchangeability matrix
$\boldsymbol{\pi}$	a stationary or initial distribution
$\boldsymbol{\tau}$	branch lengths
$\boldsymbol{\theta}$	unconstrained parameters, $\boldsymbol{\theta} \in \mathbb{R}^d$
ϕ	the constraint map, $\phi(\boldsymbol{\theta})$ the feasible parameters
$\mathbf{D}$	the observed data
$\mathcal{L}$	the scalar being minimized
H	a Hamiltonian
$\mathbf{Z}$	a hidden state path
$\mathcal{E}$	an emission family

Two conventions are worth stating because they are load-bearing later. Optimization is always *minimization*, so a likelihood-based objective is a negative log-likelihood. And $\mathcal{L}$ is a function of $\boldsymbol{\theta}$, the unconstrained vector, not of the parameters a person names—the two are connected by ϕ and Section 2.1 explains why the distinction is not cosmetic.

2 The Shape All Three Problems Share

Each problem class below couples a *combinatorial* search over a structure to a *continuous* optimization of parameters given that structure. The two are not interleaved steps of one optimizer: a structural move changes what the parameter vector *means* and how long it is, so it constructs a new objective rather than taking a step inside an existing one. Everything else in this document follows from that division.

2.1 The Continuous Optimization Abstraction

Let the objective be a differentiable scalar $\mathcal{L}(\boldsymbol{\theta})$ over an unconstrained vector $\boldsymbol{\theta} \in \mathbb{R}^d$, with a map ϕ carrying $\boldsymbol{\theta}$ to the constrained parameters a model is stated in: positive branch lengths through a logarithm, a distribution over k states through a softmax on $k-1$ free values, a positive scale through a logarithm.

Why the map and not a projection. An optimizer that must be *stopped* from leaving the feasible set will eventually leave it, and a constrained parameter that reaches its boundary has no interval around it worth reporting. Constraining by construction removes both failures: every point in $\mathbb{R}^d$ is feasible, and the gradient $\nabla_{\boldsymbol{\theta}}\mathcal{L}$ is defined everywhere.

Why the map must be injective. Where a reparameterization leaves $\mathcal{L}$ unchanged, the direction along it is not estimable: the observed information is singular and an interval is undefined. A softmax on k free values has exactly this flat direction—adding a constant to every value changes nothing—which is why $k-1$ are used. Removing the freedom is the constraint map's job, not the optimizer's.

What follows for inference. Standard errors come from the observed information at the optimum, pushed through ϕ by the delta method: $\mathrm{Var}(\phi(\boldsymbol{\theta})) \approx J\Sigma J^{\top}$ with J the Jacobian of ϕ. This is an *asymptotic* statement, exact only in the large-sample limit, which is why Section 6 makes realized interval coverage a measurement rather than an assumption.

2.2 The Discrete Search as a Decision Process

The structural search is a Markov decision process [25]: the state is the current structure with its fitted parameters, an action is a move from the neighbourhood $\mathcal{N}(\cdot)$, and the reward is the improvement in the objective. An episode ends on a step budget or when no move improves the score—a property of the state, and the same stopping rule the greedy baseline obeys, which is what makes the two comparable. Because the reward is a difference, the undiscounted return telescopes,

$$G = \sum_{t=0}^{T-1} R_t = \sum_{t=0}^{T-1} \big[\mathcal{L}(s_t) - \mathcal{L}(s_{t+1})\big] = \mathcal{L}(s_0) - \mathcal{L}(s_T), \tag{1}$$

the improvement between the first and last state whatever path joined them. That is what licenses an undiscounted return—a discount would prefer improvement found early over the same improvement found late—and it is why a truncation landing between a sacrifice and its payoff teaches the opposite of the truth, so a truncated episode is recorded as one.

The policy is a softmax over the scores of the *available* actions, $\pi(a \mid s) \propto \exp f(s,a)$ for $a \in \mathcal{N}(s)$, so a neighbourhood of any size is admissible. A scorer with a single weight on the improvement a move buys is an inverse temperature, and greedy search is its zero-temperature limit; what such an agent can learn is exactly how much it should accept a worsening move to reach a better optimum, which is the credit-assignment question this search makes sharp. It is trained by the score-function estimator with a baseline,

$$\nabla J = \mathbb{E}\Big[\sum_{t} \nabla \log \pi(a_t \mid s_t)\,\big(G_t - b(s_t)\big)\Big], \qquad G_t = \sum_{u \geq t} R_u, \tag{2}$$

where a baseline b independent of the sample it is subtracted from leaves the estimator unbiased and is there to reduce its variance, not its bias: G_t is a sum of objective differences whose scale is the problem's, so an uncentred gradient varies by orders of magnitude between instances. The comparison against the greedy baseline is at a matched budget of objective evaluations per decision, never in wall-clock time.

3 Problem Statement: Phylogenetic Inference

3.1 The model

Character evolution along a branch is a continuous-time Markov chain with rate matrix $\mathbf{Q}$; the transition probabilities over a branch of length t are the matrix exponential $\mathbf{P}(t) = \exp(\mathbf{Q}t)$ [8]. A time-reversible model factors as $\mathbf{Q} = \mathbf{S}\,\mathrm{diag}(\boldsymbol{\pi})$ with $\mathbf{S}$ symmetric, and then satisfies detailed balance, $\pi_i \mathbf{Q}_{ij} = \pi_j \mathbf{Q}_{ji}$.

The simplest such model is Jukes–Cantor on k states: every substitution equally likely and $\boldsymbol{\pi}$ uniform, with $\mathbf{Q}$ scaled so that one unit of branch length is one expected substitution per site,

$$-\sum_{i \in \mathcal{A}} \pi_i \, \mathbf{Q}_{ii} = 1, \tag{3}$$

which puts the transition probabilities in closed form [8, 7]:

$$\mathbf{P}_{ij}(t) = \frac{1}{k} + \Big(\delta_{ij} - \frac{1}{k}\Big) \exp\Big(-\frac{k}{k-1}\, t\Big). \tag{4}$$

Rows sum to one, $\mathbf{P}(0) = \mathbf{I}$, and every entry tends to $1/k$ as $t \to \infty$; the general time-reversible model replaces the single rate by $\mathbf{S}$ and $\boldsymbol{\pi}$ and keeps Equation 3.

Given a topology $\mathcal{T}$ over n taxa, branch lengths $\boldsymbol{\tau}$, and an alignment $\mathbf{D}$ of L sites, sites evolve independently and the likelihood marginalizes every assignment z of states to the internal nodes:

$$p(\mathbf{D} \mid \mathcal{T}, \boldsymbol{\tau}, \mathbf{Q}) = \prod_{s=1}^{L} \sum_{z} \pi_{z_\rho} \prod_{(u \to v)} \mathbf{P}_{z_u z_v}(\tau_v), \tag{5}$$

the product over the branches $(u \to v)$ of the rooted tree with the leaves fixed by the data—$k^{\,n-2}$ terms per site for a binary tree. Summing it directly is the oracle every faster evaluation of it is checked against.

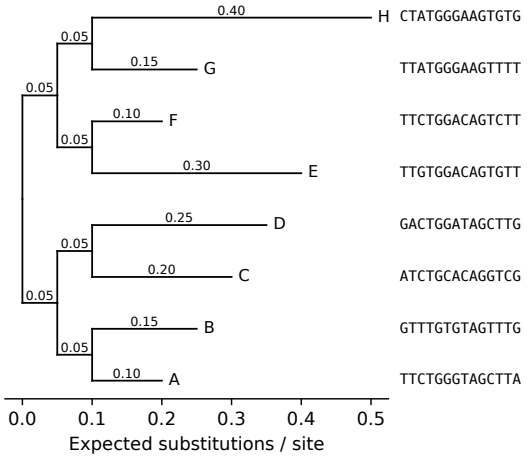

Figure 1: Simulated topology for 8 taxa under the Jukes-Cantor model (seed 20260904), branch lengths labelled in expected substitutions per site. The first 14 simulated sites are shown beside each leaf, in tree order, so the alignment the likelihood consumes can be read off against the topology that generated it.

3.2 The algorithm: simulation

The generative direction of Equation 5 is a pre-order walk: draw the root state, then each node's state from the row of its parent's state. One site is Algorithm 1; L sites are L independent runs, and the empirical substitution frequencies along any branch converge to Equation 4, which is the check a simulator is held to—against the closed form, and never against the likelihood that will later be fitted to its output.

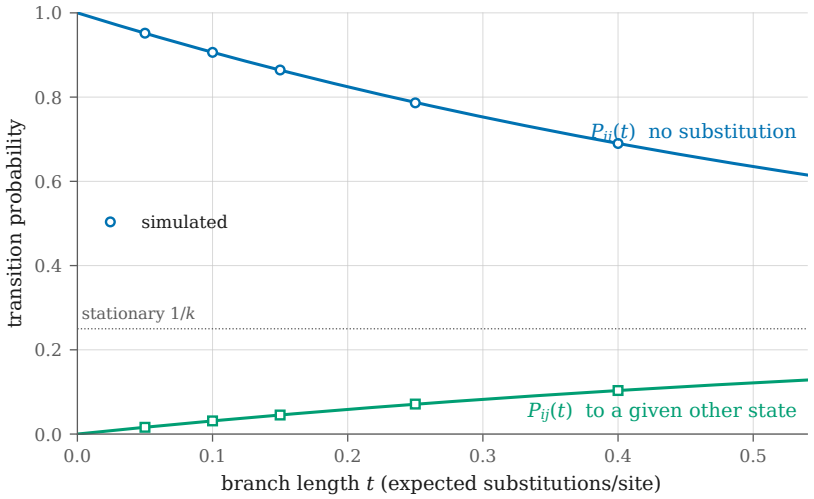

Figure 2: Closed-form Jukes-Cantor transition probabilities for k = 4 (curves) against the frequencies the simulator produced (markers), one pair per branch of the 5-branch fixture tree. Simulated at seed 20260902 over 200000 sites. Largest deviation 1.07e-03, within the fixture's declared Monte Carlo tolerance of 1e-02. The simulator is validated against this analytic form, never against the likelihood code it feeds.

Algorithm 1 Forward simulation of one site under $(\mathcal{T}, \boldsymbol{\tau}, \mathbf{Q})$

draw $z_\rho \sim \boldsymbol{\pi}$
for each non-root node v in pre-order, with parent u **do**
    draw $z_v \sim \mathbf{P}_{z_u,\cdot}(\tau_v)$
end for
return the leaf states $(z_\ell)_{\ell \in \text{leaves}(\mathcal{T})}$

3.3 The algorithm: pruning

Felsenstein's pruning evaluates that marginal in $O(nLk^2)$ rather than $O(Lk^{n-2})$, by a post-order recursion carrying a partial likelihood $L_u(i)$—the probability of the data below node u given that u is in state i:

$$L_u(i) = \prod_{v \in \text{children}(u)} \sum_{j \in \mathcal{A}} \mathbf{P}_{ij}(t_v)\, L_v(j), \tag{6}$$

with $L_u(i) = \mathbb{1}[\,u \text{ observed in state } i\,]$ at a leaf, and the site likelihood read off at the root,

$$p(\mathbf{D}_s \mid \mathcal{T}, \boldsymbol{\tau}, \mathbf{Q}) = \sum_{i \in \mathcal{A}} \pi_i\, L_\rho(i) \tag{7}$$

[8, 7]. Products of many small numbers underflow, so each node's partials are rescaled and the logarithm of the scaling accumulated separately; the rescaling must remain differentiable, since $\boldsymbol{\tau}$ and $\mathbf{Q}$ are what the continuous optimization fits.

3.4 The algorithm: discrete search

The number of unrooted binary topologies on n taxa is $(2n-5)!!$ [24], so exhaustive enumeration is an oracle at small n and nothing more. Two neighbourhoods are standard. *Nearest-neighbour interchange* (NNI) swaps the subtrees adjacent to one internal edge, giving $|\mathcal{N}_{\text{NNI}}(\mathcal{T})| = 2(n-3)$. *Subtree prune and regraft* (SPR) detaches a subtree and reattaches it at another edge, giving $2(n-3)(2n-7)$ and containing NNI. Neither is complete in one step; both are complete in the transitive closure, which for NNI is the connectivity of the associated graph [8] and is inherited by SPR because it dominates NNI.

4 Problem Statement: Potts Models in an External Field

4.1 The model

On a graph $\mathcal{G}$, each node i carries a spin $\sigma_i \in \{1, \dots, q\}$ and the energy is

$$H(\sigma) = -\sum_{\langle i,j \rangle} J_{ij}\, \delta(\sigma_i, \sigma_j) - \sum_i h_i(\sigma_i), \tag{8}$$

with couplings J_{ij} and an external field h_i [16]. The Boltzmann distribution is $p(\sigma) \propto \exp(-H(\sigma))$, and its normalizer sums over $q^{|V|}$ configurations.

4.2 Why an odd cycle is the whole story

A two-state antiferromagnet ($J < 0$) wants every edge to disagree, which is possible exactly when $\mathcal{G}$ is 2-colourable. So every bipartite lattice—a chain, a square lattice—has a trivial, unfrustrated ground state and says nothing about a hard problem. A geometry containing odd cycles is what makes the ground state non-trivial, and where a counting argument closes exactly it fixes that ground state at every size. This is the difference between a fixture that exercises an optimizer and one that does not.

4.3 The algorithms

Exact marginals come from enumeration at small sizes and from belief propagation on a tree, where it is exact; on a loopy graph it is an approximation whose accuracy is a property of the graph rather than of the implementation [12, 9]. The sum-product message from i to a neighbour j is

$$m_{i \to j}(\sigma_j) \;\propto\; \sum_{\sigma_i} \exp\big(J_{ij}\, \delta(\sigma_i, \sigma_j) + h_i(\sigma_i)\big) \prod_{l \in \partial i \setminus j} m_{l \to i}(\sigma_i), \tag{9}$$

iterated to a fixed point, with beliefs $b_i(\sigma_i) \propto e^{h_i(\sigma_i)} \prod_{l \in \partial i} m_{l \to i}(\sigma_i)$ at a node and likewise on an edge. The messages are carried in the logarithm and normalized every sweep, because a coupling of a few units on a small lattice already puts a product of exponentials past what a linear recursion holds. The Bethe free energy of the fixed point,

$$F_{\mathrm{Bethe}} = \sum_{\langle i,j \rangle} \sum_{\sigma_i, \sigma_j} b_{ij} \left(E_{ij} + \log b_{ij} \right) + \sum_i \sum_{\sigma_i} b_i \, E_i - \sum_i (d_i - 1) \sum_{\sigma_i} b_i \log b_i, \tag{10}$$

with $E_{ij} = -J_{ij}\, \delta(\sigma_i, \sigma_j)$, $E_i = -h_i(\sigma_i)$ and d_i the degree of i, equals $-\log Z$ exactly on a tree and approximates it on a loopy graph [26, 16]. A fixed point the iteration never reached is not an estimate of anything, so non-convergence is a refusal rather than a number. Sampling is by single-site Gibbs updates or by cluster moves such as Swendsen–Wang, which flip correlated regions together and so decorrelate faster near a critical point [17]. Ground states of the two-state case reduce to a maximum cut, and to a minimum cut—hence a max-flow—when the couplings are submodular; more than two states is approximated by alpha expansion, whose guarantee is a factor of two.

5 Problem Statement: Hidden Markov Models

5.1 The model

A hidden path $\mathbf{Z} = (z_1, \ldots, z_T)$ over k states evolves by an initial distribution $\boldsymbol{\pi}$ and a transition matrix $\mathbf{A}$; each position emits an observation from an emission family $\mathcal{E}$ indexed by its state. The evidence marginalizes k^T paths.

The emission family is the only part that knows what an observation is. Everything else—the recursions, the expectation step, path enumeration—needs three things from it: draw an observation given a state, score one, and re-estimate itself from posterior weights. Separating it is what lets one set of recursions serve a finite alphabet, a real observation, and a count.

A density is not a probability. For a family over a countable alphabet the evidence is a probability, so $\log p(\mathbf{D}) \le 0$. For a family over the reals it is a *density* and carries no such bound. A check written against the first is false under the second, so what may be asserted is keyed on the support the model declares.

5.2 The algorithm: forward–backward

The forward recursion is Equation 6 on a chain:

$$\alpha_t(i) = \Big[\sum_j \alpha_{t-1}(j)\, \mathbf{A}_{ji} \Big]\, \mathcal{E}_i(y_t), \qquad p(\mathbf{D}) = \sum_i \alpha_T(i). \tag{11}$$

Pruning on a caterpillar tree carrying one observed leaf per internal node *is* this recursion [7, 12]: the three problem classes are one marginalization over three structures, not three unrelated models sharing an optimizer.

5.3 The algorithm: expectation-maximization

Baum–Welch alternates a posterior over hidden states given the parameters with a re-estimate of the parameters given that posterior, and increases the likelihood at every iteration [14]. The initial distribution and the transitions re-estimate in closed form for every family; the emission re-estimate is the family's own, and *whether it is a formula* is the property that separates the families. A weighted mean is; a dispersion parameter whose score involves the digamma function is not, and its M step is itself an optimization—which is the case that says whether an interface built around closed forms was built correctly.

5.4 Two decodings, and they are different answers

Viterbi returns the single path of highest joint probability; posterior decoding returns, at each position independently, the state of highest marginal probability. Where they disagree, the posterior-decoded sequence can be a path the model assigns zero probability to, because nothing constrains consecutive choices to be compatible. They agree on almost every instance, which is exactly why an implementation that computes one and reports the other survives a test suite with no case where they diverge.

6 The Properties That Referee Each Method

An algorithm is accepted here against a property that is true independently of this implementation. The properties are not interchangeable, and which one applies is a statement about the method rather than a matter of convenience.

6.1 Exact answers, where an oracle exists

Where a quantity can be computed a second way that shares no recursion with the first, equality is asserted to a stated tolerance. Direct marginalization over k^{n-2} internal assignments referees pruning; a sum over k^T paths referees the forward recursion; exhaustive enumeration over $q^{|V|}$ configurations referees a Potts marginal; enumeration of $(2n-5)!!$ topologies referees a search. Every one is exponential, and that is what makes it an oracle rather than a second opinion. **Two implementations agreeing prove nothing if both are wrong**, so an accelerated method is pinned against the reference and never against another acceleration.

6.2 Structural invariants, where no oracle reaches

Past the sizes enumeration allows, what remains are properties the model must satisfy whatever the answer is:

- rows of a transition matrix sum to one, and $\mathbf{P}(0) = \mathbf{I}$;
- a reversible model satisfies detailed balance, $\pi_i \mathbf{Q}_{ij} = \pi_j \mathbf{Q}_{ji}$;
- the *pulley principle*: on a reversible model the likelihood is invariant to where the root is placed along a branch, so a rooted computation and its re-rooting must agree exactly;
- a gradient matches central differences, relative to the gradient's own norm—entrywise fails wherever an entry is zero, and an absolute bound does not transfer across data sizes;
- an expectation-maximization iterate increases the likelihood monotonically.

None of these establishes that a method is right. Each establishes that a specific way of being wrong did not happen, which is a weaker and more defensible claim.

6.3 Recovery, which is the acceptance test for a fit

A likelihood that increases proves the optimizer runs, not that the model is right. The acceptance test is to fit data simulated from known parameters and require the intervals to cover the truth at their nominal rate over independent replicates. Two qualifications are load-bearing. Where a model has an exact symmetry—permuting the hidden states of an HMM or the components of a mixture leaves the likelihood unchanged—recovery is stated *up to that symmetry*, and the alignment is part of the test. And coverage is measured as a function of how identifiable the instance is, because a fixture chosen where everything is well separated is a test of the fixture.

6.4 Where the likelihood has no maximum, or no curvature

Two failures are properties of models rather than defects, and both must be handled before they are discovered.

An *unbounded* likelihood has no maximum to find: put a Gaussian component's mean on a single observation and let its variance go to zero and the density there diverges. A fit that converges there was stopped by its starting point or by a floor, and which one must be knowable—so the bound is explicit, derived from the data rather than chosen, and reaching it is a refusal rather than a clamp.

A *flat* likelihood has a maximum that runs to the boundary: an overdispersed count model approaches its Poisson limit as the dispersion grows, and past the point where the overdispersion is smaller than the sampling noise on measuring it, the data cannot tell one value from another. The two failures are mirror images, and they announce themselves differently—the first as a divergence, the second as a singular information matrix—which is why each is measured rather than inferred from the other.

6.5 Agreement across implementations is a tolerance

An accelerated implementation is pinned against the reference within a declared *relative* tolerance, never bitwise. Relative, because a log-likelihood is a sum over sites and an absolute bound fixed at one size does not transfer to another. Keyed on the *lowest* precision in the comparison, because a single-precision device cannot be held to a double-precision bound, and one bound loose enough for single precision would let a broken double-precision backend pass. A discrepancy inside the tolerance is not a defect and is not corrected; one outside it is not a tolerance to loosen. The values themselves live beside the measurements they are derived from, not here.

6.6 Published constants, and asymptotic results

Some claims come from outside entirely: the $(2n-5)!!$ count, Goemans–Williamson's 0.87856 for maximum cut, alpha expansion's factor of two, the $8(\ln k + 2)$ bound on k-means++ seeding. These referee an implementation at any size. **No asymptotic result is testable at the sizes an exact oracle allows**, however: a threshold or a limit holds as the problem grows and says nothing where enumeration can check it. Such a result is reported for orientation and the per-instance property is what is asserted.

7 Relevance to the Roadmap

The three problem classes are not three applications. They are chosen so that each abstraction is tested against something that is not the model it was extracted from: a likelihood engine justified only by trees would be a phylogenetics library, and an optimizer justified only by an HMM would encode a chain.

The order the milestones take follows the dependency between the properties above. Simulation and its ground truth come first, because every later claim is checked against data whose generating parameters are known. The exact evaluators come next, because they are the oracles the accelerated ones are pinned to. Continuous optimization follows, since recovery is the acceptance test and needs both. Discrete search and learned proposals come last, because their acceptance criterion—reaching the optimum enumeration identifies—only exists once enumeration does.

References

[1] Shun-ichi Amari. *Information Geometry and Its Applications.* Springer, Tokyo, 2016.

[2] Richard E. Blahut. *Algebraic Codes for Data Transmission.* Cambridge University Press, Cambridge, 2003.

[3] Jim Blandy, Jason Orendorff, and Leonora F. S. Tindall. *Programming Rust: Fast, Safe Systems Development.* O'Reilly Media, 2nd edition, 2021.

[4] Randal E. Bryant and David R. O'Hallaron. *Computer Systems: A Programmer's Perspective.* Pearson, 3rd edition, 2015.

[5] Phillip Compeau and Pavel Pevzner. *Bioinformatics Algorithms: An Active Learning Approach.* Active Learning Publishers, 2015.

[6] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to Algorithms.* MIT Press, Cambridge, Massachusetts, 4th edition, 2022.

[7] Richard Durbin, Sean R. Eddy, Anders Krogh, and Graeme Mitchison. *Biological Sequence Analysis: Probabilistic Models of Proteins and Nucleic Acids.* Cambridge University Press, Cambridge, 1998.

[8] Joseph Felsenstein. *Inferring Phylogenies.* Sinauer Associates, Sunderland, Massachusetts, 2004.

[9] Brendan J. Frey. *Graphical Models for Machine Learning and Digital Communication.* MIT Press, Cambridge, Massachusetts, 1998.

[10] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning.* MIT Press, Cambridge, Massachusetts, 2016.

[11] Wen-mei W. Hwu, David B. Kirk, and Izzat El Hajj. *Programming Massively Parallel Processors: A Hands-on Approach.* Morgan Kaufmann, 4th edition, 2022.

[12] Daphne Koller and Nir Friedman. *Probabilistic Graphical Models: Principles and Techniques.* MIT Press, Cambridge, Massachusetts, 2009.

[13] Maxim Lapan. *Deep Reinforcement Learning Hands-On.* Packt Publishing, 2nd edition, 2020.

[14] David J. C. MacKay. *Information Theory, Inference, and Learning Algorithms.* Cambridge University Press, Cambridge, 2003.

[15] Robert C. Martin. *Clean Code: A Handbook of Agile Software Craftsmanship.* Prentice Hall, 2008.

[16] Marc Mézard and Andrea Montanari. *Information, Physics, and Computation.* Oxford University Press, Oxford, 2009.

[17] M. E. J. Newman and G. T. Barkema. *Monte Carlo Methods in Statistical Physics.* Oxford University Press, Oxford, 1999.

[18] Michael A. Nielsen and Isaac L. Chuang. *Quantum Computation and Quantum Information.* Cambridge University Press, Cambridge, 10th anniversary edition, 2010.

[19] Jorge Nocedal and Stephen J. Wright. *Numerical Optimization.* Springer, 2nd edition, 2006.

[20] Antonio Ortega. *Introduction to Graph Signal Processing.* Cambridge University Press, Cambridge, 2022.

[21] Lior Pachter and Bernd Sturmfels, editors. *Algebraic Statistics for Computational Biology.* Cambridge University Press, Cambridge, 2005.

[22] Simon J. D. Prince. *Understanding Deep Learning*. MIT Press, Cambridge, Massachusetts, 2023.

[23] Sebastian Raschka. *Build a Large Language Model (From Scratch)*. Manning Publications, 2024.

[24] Kenneth H. Rosen. *Discrete Mathematics and Its Applications*. McGraw-Hill, 8th edition, 2019.

[25] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, Cambridge, Massachusetts, 2nd edition, 2018.

[26] Jonathan S. Yedidia, William T. Freeman, and Yair Weiss. Constructing free-energy approximations and generalized belief propagation algorithms. *IEEE Transactions on Information Theory*, 51(7):2282–2312, 2005.

A Reference Taxonomy

Grouped as the repository groups them, so one routing table serves the code and the documents alike.

Infrastructure (build, structure, and speed). Software craft: Martin [15], Blandy et al. [3]. Systems and hardware, including memory hierarchies and parallel kernels: Bryant and O'Hallaron [4], Hwu et al. [11].

Optimization (discrete and continuous). Algorithms and discrete mathematics: Cormen et al. [6], Rosen [24]. Numerical optimization: Nocedal and Wright [19]. Probabilistic inference and message passing: MacKay [14], Koller and Friedman [12], Frey [9], Ortega [20]. Statistical physics and Monte Carlo: Mézard and Montanari [16], Newman and Barkema [17]. Information geometry: Amari [1]. Learning and reinforcement learning: Goodfellow et al. [10], Prince [22], Sutton and Barto [25], Lapan [13], Raschka [23].

Application (the science). Phylogenetics: Felsenstein [8], Durbin et al. [7], Compeau and Pevzner [5], Pachter and Sturmfels [21]. Information and coding: Blahut [2], with Nielsen and Chuang [18] as background only.